

- Gitlab:
 - Sistema de versionado y ramas
 - Tableros: abierta > en progreso > cerrado
 - CI/CD
 - Estimación de tareas
 - Discord
 - Metro Retro
 - Zoom
 - Excel
 - OneNote
-

- Visual studio code
 - IntelliJ
 - MySQL workbench
 - Linux CLI / Bash / aws cli
 - Selenium IDE
-

- - Html
 - Css
 - Sass
 - Javascript
 - NodeJS
 - Npm
 - React
 - Jest

- Librerías externas (sweet-alert, formik, react-icons, react-leaflet, react-jwt, bootstrap, react-date-picker, axios, swiper, react-share, react-select, react-glider)

-

- IntelliJ IDEA.
- Java.
- Spring Boot.
- Maven.
- ORM Hibernate.
- Spring MVC.
- Spring Data JPA.
- Spring Security.
- JWT.
- Swagger.
- Bases de datos Relacionales(MySQL).
- Postman.
- Log4j.
- Java mail Sender.
- Html
- Lombok

-

- MySQL
- Amazon RDS

-

- Selenium
- Jest
- React Testing Library

-

- AWS management console
 - Terraform
 - linux (ubuntu)
 - nginx
 - NoIP
-

El proyecto está íntegramente alojado en gitlab, es necesario clonar el repositorio para obtener todos los archivos. Se utiliza una rama personal por cada integrante del equipo, en la cual se trabaja hasta completar una tarea, momento en el que se hace merge sobre la rama principal.

El Frontend ha sido confeccionado usando la librería de React que nos permite gestionar nuestra página en componentes reutilizables. La organización de carpetas se ha establecido de la siguiente forma

- test
- Components
- Context
- FuncionesJS
- Helpers
- Pages
- Routing

En test se encuentran los test realizados con Jest y React Testing Library para cumplir la cobertura desde el lado del Front. Components está conformado por pequeños componentes que se han reutilizado en toda la aplicación. Allí mismo se creó un Layout que permite que el header y el footer se renderice en todas las páginas de la aplicación Context permite que de acuerdo al rol que cumple el usuario se renderice ciertas páginas a las cuales tiene permiso. FuncionesJS posee funciones que han sido reutilizadas en diferentes páginas a lo largo del proyecto. Helpers contiene la URL de la API Pages tiene las 7 páginas obligatorias que el producto demandaba y una opcional llamada NotFound que se

renderiza en caso que el usuario no posea permisos o el link que ha solicitado navegar no exista. Por último, Routing contiene todas las rutas de navegación de la aplicación desarrolladas con React Router Dom.

Esta disposición de carpetas permitió mantener un espacio de trabajo limpio y organizado, haciendo que trabajar en las tareas sea más sencillo y, por lo tanto, que la aplicación web sea escalable sin ninguna dificultad. A la hora de repartir tareas se tuvo en cuenta de evitar en lo posible que ambos integrantes trabajen en el mismo archivo para evitar conflictos al momento de hacer merge, lo que logró que la integración del código se haga de manera rápida y fluida, aunque si uno de los dos tenía dificultades para avanzar con su tarea, sin duda alguna ambos trabajaban en el mismo archivo para sacar la tarea adelante.

Se dispuso de hacer commits con pequeñas funcionalidades para que se pueda ver el progreso del otro así como también tener la versión más actual del código. En el caso de que una funcionalidad tuviese errores, se limitaba solo hacer push a la rama de uno para no perjudicar el trabajo del compañero.

Tanto el feedback del equipo Scrum como las sugerencias de los demás compañeros del equipo se fue registrando en un OneNote en forma de lista de tareas pendientes, donde se iba tachando a medida que se corregían los errores.

Respecto a la parte técnica, la lógica fue desarrollada usando el lenguaje JSX y los estilos, CSS y Sass. Se hizo uso de librerías externas tanto para facilitar el trabajo de programación como para darle las funcionalidades que requería el producto. Entre las librerías más destacadas se encuentra la de react-calendar utilizada en 3 páginas diferentes, en donde cada una el calendario cumplía una función distinta, y Formik para el manejo de formularios para crear reservas y crear propiedades.

Para resguardar la seguridad de la página a partir de la lectura del token que proviene del Back End se configuró la lógica para renderizar ciertas páginas según el rol que tenga el usuario, pudiendo tener este el rol de administrador, rol de usuario registrado o ningún rol asignado.

Sin perder de vista de otorgar la mejor experiencia de usuario, la página cuenta con animaciones en las cards y botones así como SweetAlerts en el caso que se le quiere comunicar al usuario un error, advertencia o éxito es algún procedimiento.

El proyecto está estructurado según el Modelo Vista Controlador separando las clases según las entidades sus repositorios y sus controladores donde se aloja la lógica necesaria para darle cumplimiento a las issues pedidas, mapeamos las entities que vienen de una base de datos SQL usando jpa e hibernate ORM para crear su repositories con sus respectivas queries para darle cumplimiento al CRUD y otras funcionalidades pedidas, para el seguimiento del funcionamiento de la aplicación se usaron excepciones Globales y log4j para el manejo de errores, se usa spring security para proteger la aplicación junto a jwt añadiendo la seguridad requerida a cada endpoint y habilitando los CORS para que puedan consumir la api desde otros servicios. para saber cómo funciona cada endpoint se puede ver su documentación en [_____](#) También se usó java mail sender para el envío de email

y html para generar una plantilla de correo que se enviará a cada usuario junto a el link de confirmación

Informe final de testing.

La base de datos está alojada sobre AWS. Para poder trabajar sobre la misma es necesario conectar el workbench utilizando los datos del endpoint y las credenciales. Se crearon 2 scripts: uno que genera las tablas y las relaciona; y el otro que hace la carga inicial de los datos que se muestran en la página web. Los mismos se encuentran en el repositorio de gitlab. NO se deben utilizar, ya que borran todos los registros. En este punto se recomienda no ingresar datos de manera manual a través del workbench, ya que de eso se encarga la api de backend. Además, no se pueden ingresar usuarios ya que la contraseña necesita ser encriptada por la api. Para migrar la base de datos es necesario hacer un dump, el cual se crea al exportar todos los registros de la base de datos.

La infraestructura está alojada íntegramente en amazon web services. La arquitectura y el flujo de las redes están en el repositorio de gitlab. Se puede acceder a la consola web de amazon utilizando las credenciales provistas por la academia. Para desplegar la infraestructura se utilizan scripts de terraform. Hay dos tipos, un script que contiene toda la infraestructura el cual sirve para migrar todo hacia una zona diferente. Y también hay scripts individuales que sirven para agregar, quitar o reparar cada una de las partes de la arquitectura. La aplicación está alojada en tres servidores ec2. Uno funciona como servidor api, y los otros dos funcionan como servidores web. El acceso a los mismos se puede hacer desde la consola web de amazon. También se puede acceder mediante SSH, utilizando la key privada del grupo 6. Para actualizar los servidores utilizamos comandos bash. Se puede seguir este tutorial para hacerlo: [_____](#). Para agregar nuevos productos es necesario agregar las imágenes al bucket S3. El acceso al sitio web es mediante un dns gratuito, el cual redirecciona a la zona alojada en el servicio route53 de amazon.
